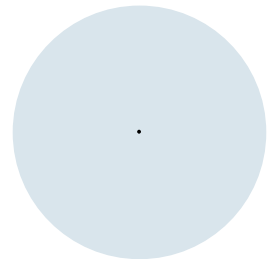


Zeichendarstellung



Zeichendarstellung

- Computer kennt keine Zeichen, nur (Binär-)Zahlen
- Daher Abbildung Zeichen auf Zahl
 - Z.B. A = 0, B = 1, ..., Z = 26, a = 27, b = 28, usw
 - (hier dezimal dargestellt)
- Im Computer werden dann Zahlen gespeichert
 - ABBA = 0 1 1 0
- Heutige Zeichenkodierungen basieren auf dem **American Standard Code for Information Interchange** (kurz **ASCII**), welcher erstmals 1963 vorgestellt wurde
- Buchstabe A hat hier den Wert 65 (Dezimal)

ASCII 7-bit (0-127)

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174	|	
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177		DEL

Source: www.LookupTables.com

ISO-8859-15 8-bit

0-127: wie ASCII

128-255: Erweiterung alle Sonderzeichen der deutschen, französischen, estnischen und finnischen Sprache

<i>PAD</i>	<i>HOP</i>	<i>BPH</i>	<i>NBH</i>	<i>IND</i>	<i>NEL</i>	<i>SSA</i>	<i>ESA</i>	<i>HTS</i>	<i>HTJ</i>	<i>VTS</i>	<i>PLD</i>	<i>PLU</i>	<i>RI</i>	<i>SS2</i>	<i>SS3</i>
<i>DCS</i>	<i>PU1</i>	<i>PU2</i>	<i>STS</i>	<i>CCH</i>	<i>MW</i>	<i>SPA</i>	<i>EPA</i>	<i>SOS</i>	<i>SGCI</i>	<i>SCI</i>	<i>CSI</i>	<i>ST</i>	<i>OSC</i>	<i>PM</i>	<i>APC</i>
<i>NBSP</i>	ı	¢	£	€	¥	Š	§	š	©	ª	«	¬	ŠHY	®	—
°	±	²	³	Ž	µ	¶	·	ž	¹	º	»	Œ	œ	Ÿ	ı
À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

Probleme bei 8-bit

- Nur bestimmte Sonderzeichen darstellbar.
- Für bestimmte Sprachfamilien unterschiedliche Zusatzzeichen 128-255
- hier die Zeichen 176-255 in ISO-8859-15 (westeuropäisch) und in ISO-8859-7 (griechisch)

°	±	²	³	Ž	μ	¶	·	ž	¹	º	»	Œ	œ	Ÿ	¿
À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

	242	207	211	211	201	209
KOI-8-R	Р	О	С	С	И	Я
ISO-8859-1	ò	ï	ó	ó	é	ñ

Re: New Harry Potter Book

От: Svetlana Pletneva <sveta@example.ru>

Кому: claudette@example.fr

Тема: Re: New Harry Potter book

Hey Claudette,

That would be amazing – we cannot get new Harry Potter book here in Russia yet! Here is my address; it is in Russian alphabet so please copy it carefully:

Россия, Москва, 119415
пр.Вернадского, 37,
к.1817-1,
Плетневой Светлане

Thanks!

Sveta

Re: New Harry Potter Book

From: Svetlana Pletneva <sveta@example.ru>

To: Claudette Toussant

Subject: Re: New Harry Potter book

Hey Claudette,

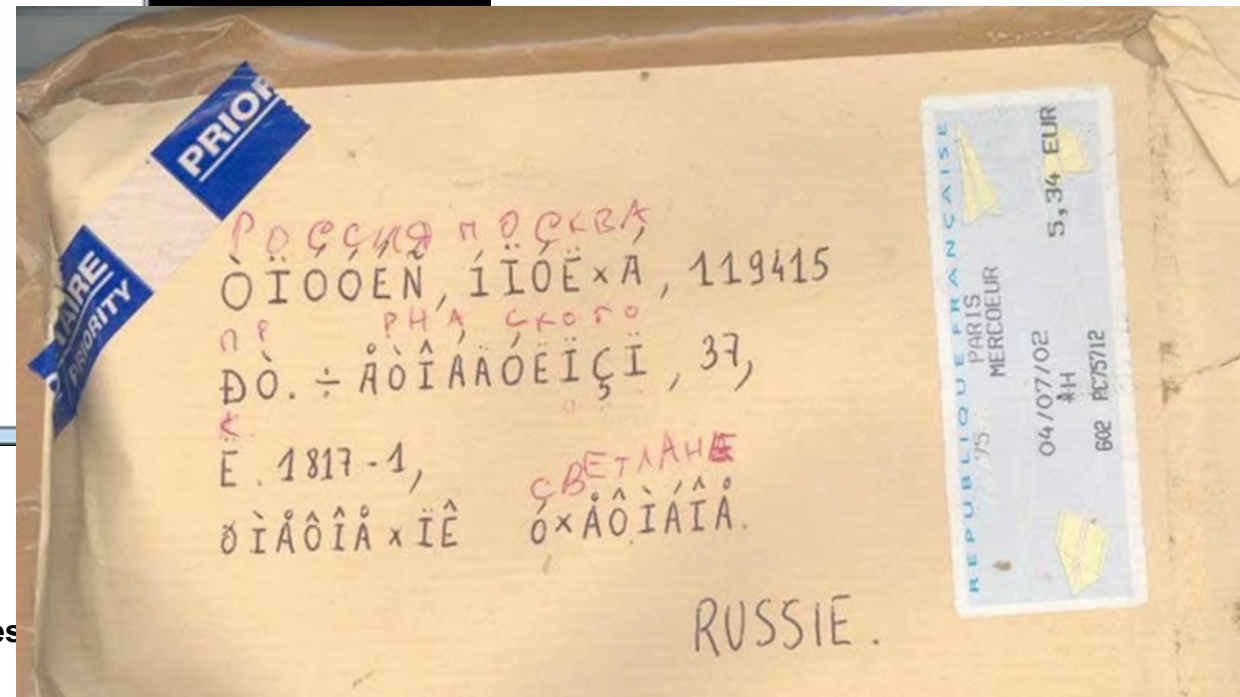
That would be amazing – we cannot get new Harry Potter book here in Russia yet! Here is my address; it is in Russian alphabet so please copy it carefully:

òïóóéñ, ïïóë×á, 119415
ðò.÷àòíáäóëïçì, 37,
ë.1817-1,
ðìäôîâ×îë ó×äòíâîâ

Thanks!

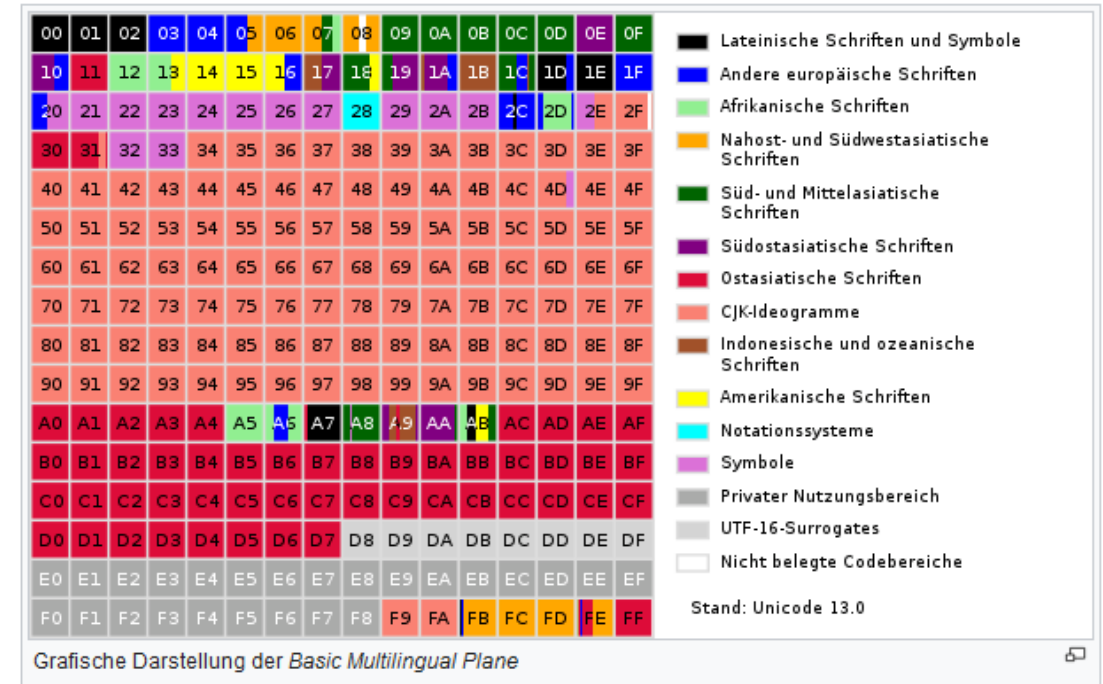
Sveta

["Plain Text" by Dylan Beattie @ NDC Oslo 2021](#)



Unicode

- 16-bit-Darstellung, also $2^{16} = 65.536$ mögliche Zeichen in der "Basic Multilingual Plane"
- beinhalten alle Zeichen lebendiger Sprachen
- 0 bis 127 wie bei ASCII
- später noch um zusätzliche Ebenen erweitert, z.B. historische Sprachen, Emojis, ... (dafür braucht man mehr als 16-bit)
- Wir betrachten hier Unicode als 16-bit-Darstellung



Zeichentypen (Characters)

- Java: char
- 16 Bit Unicode
- Auch als 2 Byte vorzeichenlose Ganzzahl verwendbar

Variablen

- **Benannte** und **typisierte** Speicherbereiche, die Werte des angegebenen Typs enthalten können.
- Variablen müssen vor ihrer Benutzung deklariert werden:

```
int age;  
double averageGrade;  
char nextLetter;
```

(erst Typ, dann Name)

- Variablen müssen initialisiert werden, z.B. durch Zuweisung eines initialen Werts als *Literal* (mehr dazu gleich)

Literale

- Typisierte Konstanten, die u.a. verwendet werden können, um Variablen zu initialisieren.

Typ	Beispielliterale	evtl. Suffix
byte	127, -128	
short	32456	
int	60273	
long	13292367454L	L, l
float	87.363f	F, f
double	37.266, 37.266D	(default), D, d, e3
char	'A', 'a'	

← klein L besser nicht verwenden,
Verwechslungsgefahr mit 1

Beispiel Deklaration und Initialisierung mit Literal

```
int intNumber;  
long longNumber;  
boolean isEmpty;  
double averageGrade;
```

Deklaration

```
intNumber = 42;  
longNumber = 2015L;  
averageGrade = 2.5;
```

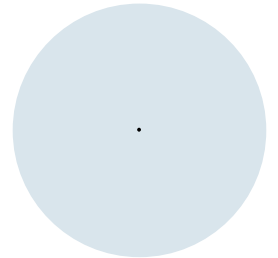
Literal-Zuweisung

```
char key = 'k';
```

Neu!

jsHELL>

Strings



Strings (Zeichenketten)

- String ist **kein primitiver** Typ, sondern ein sog. **Referenz-Typ** (mehr dazu später)
- **String-Literale** werden mit `""` eingefasst
- Deklaration wie üblich: Typ Variablenname;

```
String text;  
text = "Hello world!";
```

oder kürzer

```
String text = "Hello world!";
```

- Es gibt auch den leeren String, also ein String der Länge 0 ohne Inhalt:

```
String empty = "";
```

jsHELL>

String-Operator +

- Zwei Strings kann man mit + aneinanderhängen (Operation "Konkatenation")

```
String greeting = "Hello";  
String text = greeting + " world!";
```

- Das geht auch über mehrere Zeilen

```
System.out.println("Hallo " +  
    "Welt!");
```

- Man kann auch andere Datentypen mit + zu Strings hinzufügen (werden dann automatisch umgewandelt)

```
int isEmpty = 42;  
System.out.println("Wert von zahl: " + zahl);
```

jshe11>



code03.
String
Basics

Besondere Zeichen

- Will man " innerhalb eines Strings verwenden, so muss man \ voranstellen ("escapen")

```
System.out.println("Mein Name ist \"Hase\");
```

Mein Name ist "Hase"

- Es gibt Sonderzeichen wie \n (neue Zeile), \t (Tabulator) und \\ (einfacher Backslash)

```
String twoLines = "Zeile1\nZeile2";
```

Zeile1
Zeile2

```
String tempFile = "C:\\temp\\tempfile.txt";
```

C:\temp\tempfile.txt



code03.
String
Escape

Mehrzeilige Strings

- Strings über mehrere Zeilen kann man über \n und + zusammenbauen:

```
String htmlString = "<html>\n" +  
    "    <body>\n" +  
    "        <p>Hello, world</p>\n" +  
    "    </body>\n" +  
    "</html>\n";
```

- Seit Java 15 gibt es jedoch die praktischen Mutli-Line-String-Literale, die mit """" umschlossen werden:

```
String htmlStringDefinedAsMultiline = """  
    <html>  
        <body>  
            <p>Hello, world</p>  
        </body>  
    </html>  
""";
```

erste Einrückung

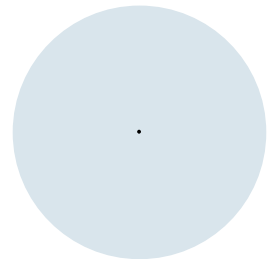
Java nimmt die erste
Einrückung als Referenz
und schneidet das hier
automatisch ab.



code03.

MultiLine
Strings

Wahrheitswerte



Problem Motivation

- Manchmal ist ergebnis einer Berechnun gkeine Zahl sondern eine logische Aussage
- Mindestparkzeit überschritten?
- Erwachsen?
- Könnte man mit Zahlen codieren, JAVA bietet aber speziellen Datentyp dafür an.

Booleans

- **Booleans** = logische (boolesche) Werte
 - Java: boolean
 - **Wahrheitswert true/false** (wahr/falsch)
 - Logisch miteinander verknüpfbar
A und B, A oder B, nicht A etc.
- Im Speicher als 4 Byte int, von dem nur 1 Bit genutzt wird
 - Speicherineffizient: 1 Bit genutzt, 32 Bit belegt
 - Einfach+schnell, da Prozessoren auf 32 Bit-Daten optimiert sind

Variablen

- **Benannte** und **typisierte** Speicherbereiche, die Werte des angegebenen Typs enthalten können.
- Variablen müssen vor ihrer Benutzung deklariert werden:

```
int age;  
double averageGrade;  
char nextLetter;  
boolean isEmpty;
```

(erst Typ, dann Name)

- Variablen müssen initialisiert werden, z.B. durch Zuweisung eines initialen Werts als *Literal* (mehr dazu gleich)

Literale

- Typisierte Konstanten, die u.a. verwendet werden können, um Variablen zu initialisieren.

Typ	Beispielliterale	evtl. Suffix
byte	127, -128	
short	32456	
int	60273	
long	13292367454L	L, l
float	87.363f	F, f
double	37.266, 37.266D	(default), D, d, e3
char	'A', 'a'	
boolean	false, true	

← klein L besser nicht verwenden,
Verwechslungsgefahr mit 1

Beispiel Deklaration und Initialisierung mit Literal

```
int intNumber;  
long longNumber;  
boolean isEmpty;  
double averageGrade;
```

Deklaration

```
intNumber = 42;  
longNumber = 2015L;  
isEmpty = true;  
averageGrade = 2.5;
```

Literal-Zuweisung

```
char key = 'k';  
boolean isFull = false;
```

in einem Schritt

Beispiel inkompatible Zuweisung

```
int age;  
age = 22.5;
```

↑ ↑
Typ int ≠ Typ double → **Compilerfehler!**

```
char key = 'a';  
boolean isEmpty = key;
```

↑ ↑
Typ boolean ≠ Typ char → **Compilerfehler!**

Hinweis: Manche Zuweisungen unterschiedlicher Typen sind aber möglich,
z.B. einen int-Wert an eine long-Variable. Mehr dazu später!

Logische Operatoren

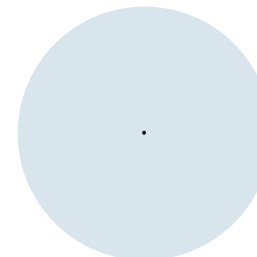
- Operanden vom Typ `boolean`
- Ergebnis vom Typ `boolean`

Operator	Bedeutung	Stelligkeit	Position	Beispiel
!	logisches NOT	unär	Präfix	!a
&	logisches, bitweises AND	binär	Infix	a & b
&&	logisches AND	binär	Infix	a && b
	logisches, bitweises OR	binär	Infix	a b
	logisches OR	binär	Infix	a b



code03.
LogicalOperators

Primitive Datentypen in Java



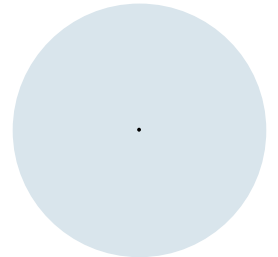
Primitive Datentypen in Java

Typ	Länge	Kleinste darstellbare Zahl	Größte darstellbare Zahl
byte	8 Bit	-128	127
short	16 Bit	-32.768	32.767
int	32 Bit	-2.147.483.648	2.147.483.647
long	64 Bit	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
float	32 Bit	ca. $(-2) \times 2^{127}$	ca. 2×2^{127}
double	64 Bit	ca. -10^{308}	ca. 10^{308}
char	16 Bit	Unicode-Zeichen	
boolean	32 Bit	false	true

primitive Datentypen

- beinhalten einen Wert ("Skalar")
- können einfach und effizient auf ein bis zwei Speicherregister abgebildet werden

Typisierte Programmiersprachen



Nicht-typisierte Sprachen

- Operationen auf Speicherwerten (Bits) ohne Typ, z.B. Assembler

```
mov ah, 0x0e ; Weise dem oberen Byte des  
              Registers a den Wert 14 zu.  
mov al, '!'   ; Weise dem unteren Byte den  
              Wert des Zeichens '!' zu.
```

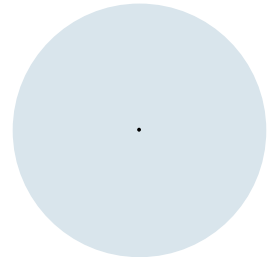
Dynamisch typisierte Sprachen

- Jeder Wert/jede Variable hat einen Typ, der dynamisch zur Laufzeit des Programms feststeht und sich ändern kann
- Beispiel JavaScript:
var value = "Hallo"; ← Typ String
value = 42; ← Typ Zahl, funktioniert
value.substring(4); ← Laufzeit-Fehler
(substring() nicht für Typ Zahl vorhanden)

Statisch typisierte Sprachen

- Java ist eine **statisch typisierte Sprache**
- Der Typ eines Wertes/einer Variable wird zur Entwicklungszeit festgelegt und kann nicht mehr geändert werden
- Der Compiler stellt Typsicherheit her
- Beispiel:
String value = "Hallo"; ← Typ String
value = 42; ← Compilerfehler
"required: String, got: int"

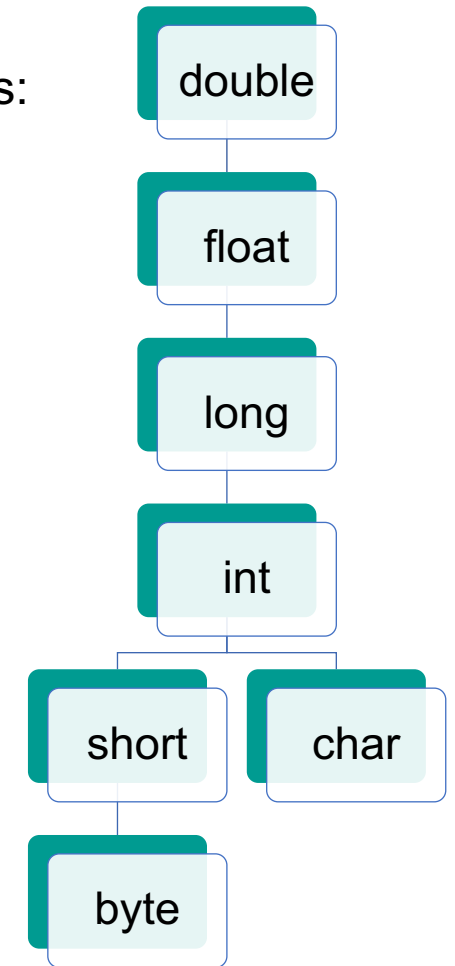
Typhierarchie und Typumwandlung



Hierarchie der primitiven Datentypen

- Wertebereich eines engeren Typs ist **kleiner** als Wertebereichs eines weiteren Typs:
 - Ganzzahltypen < Gleitkommazahltypen
 - Ganzzahltypen: byte < short < int < long
 - Gleitkommazahltypen: float < double
 - char < int

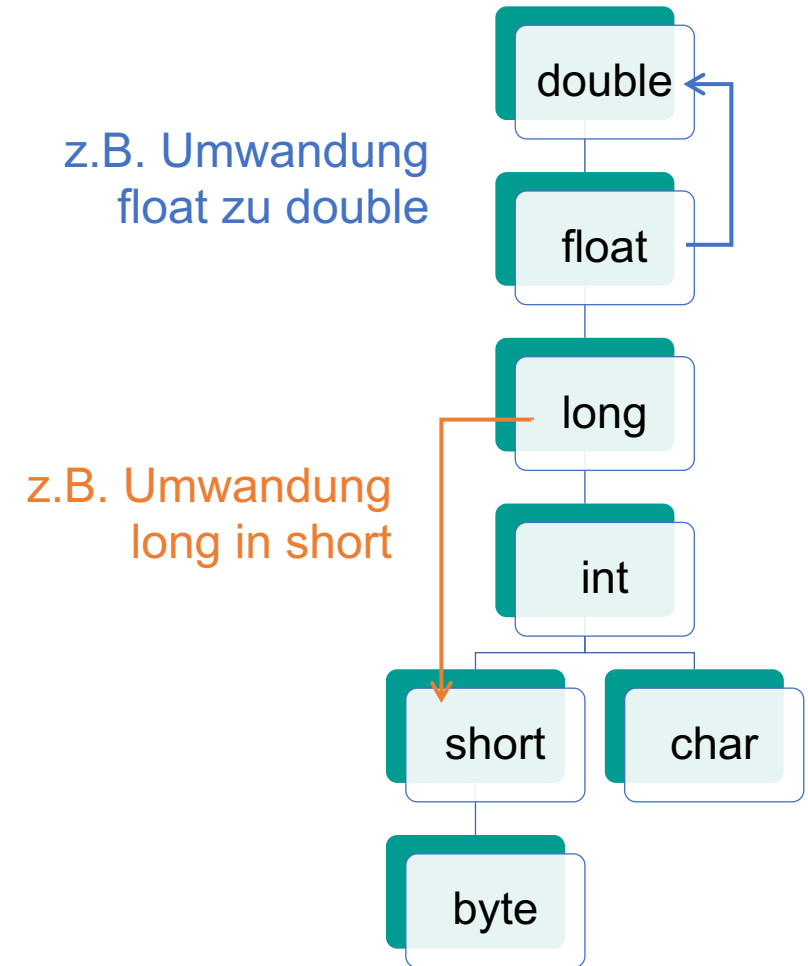
Typ	Länge	von	bis
byte	8 Bit	-128	127
short	16 Bit	-32.768	32.767
int	32 Bit	-2.147.483.648	2.147.483.647
long	64 Bit	-9.223.372.036.854.775.808	9.223.372.036.854.775.807
float	32 Bit	ca. $(-2) \times 2^{127}$	ca. 2×2^{127}
double	64 Bit	ca. -10^{308}	ca. 10^{308}
char (als Zahl)	16 Bit	0	65535



Typumwandlungen

In zwei Richtungen:

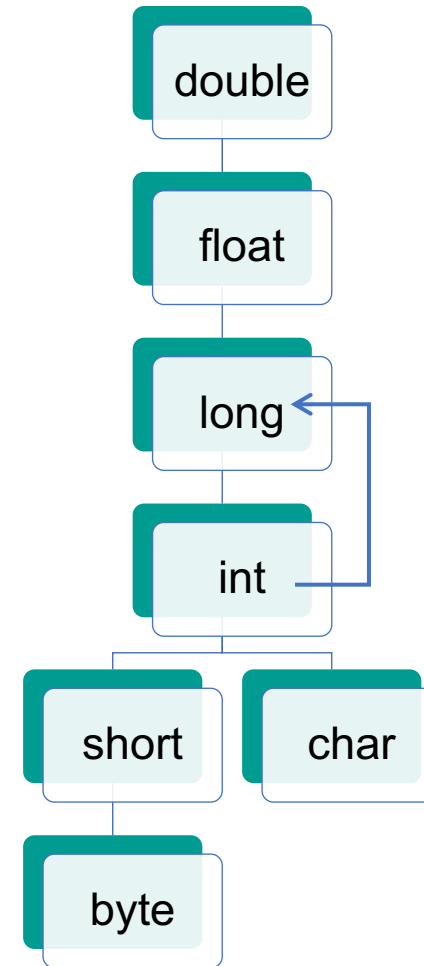
- aufwärts ("**upcast**"): enger Typ zu weiterem Typ
- abwärts ("**downcast**"): weiter Typ zu engerem Typ
- Kein anderer Typ kann in *boolean* konvertiert werden und *boolean* nicht in andere Typen



Upcast (implizit)

- Werte engerer (niedriger) Typen können Variablen weiteren (höheren) Typs zugewiesen werden
- Java konvertiert automatisch

```
int intNumber = 42;  
long longNumber;  
  
longNumber = intNumber;
```



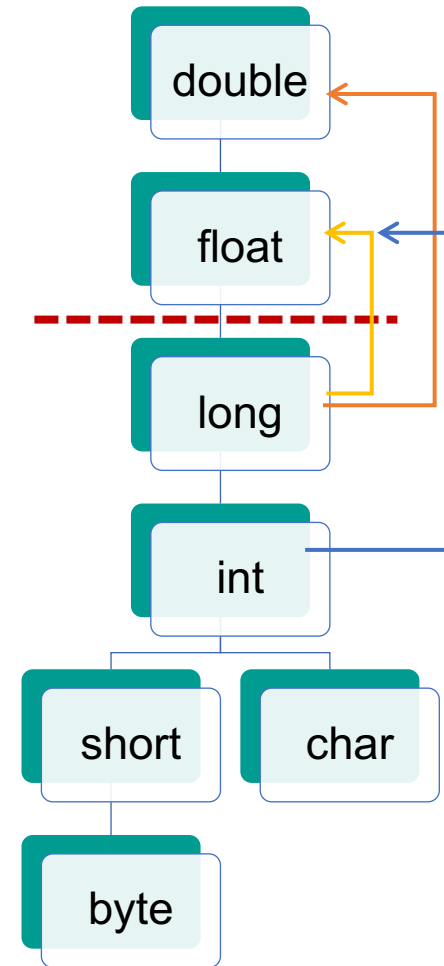
jshell>



code03.
Implicit
Upcast

Präzisionsverlust bei upcast

- Wandlung von Ganzzahl in Gleitkommazahl kann zu **Präzisionsverlust** führen!
- Betrifft nur upcasts, die die gestrichelte Linie überschreiben, z.B.:
 - `int` zu `float`
 - `long` zu `float`
 - `long` zu `double`



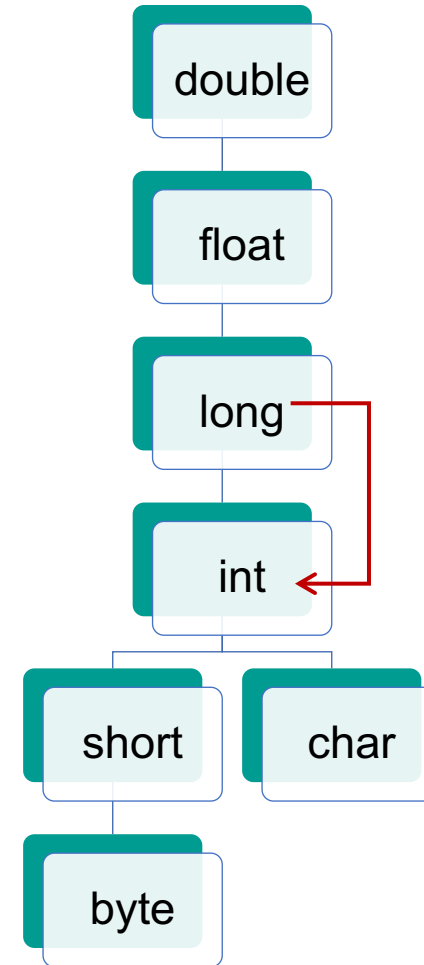
code03.
FloatUpcast
PrecisionLoss

Downcast

- Java konvertiert nicht automatisch
- Compilerfehler

```
int intNumber;  
long longNumber = 42L;
```

```
intNumber = longNumber;
```



jshell>



code03.
Implicit
Downcast

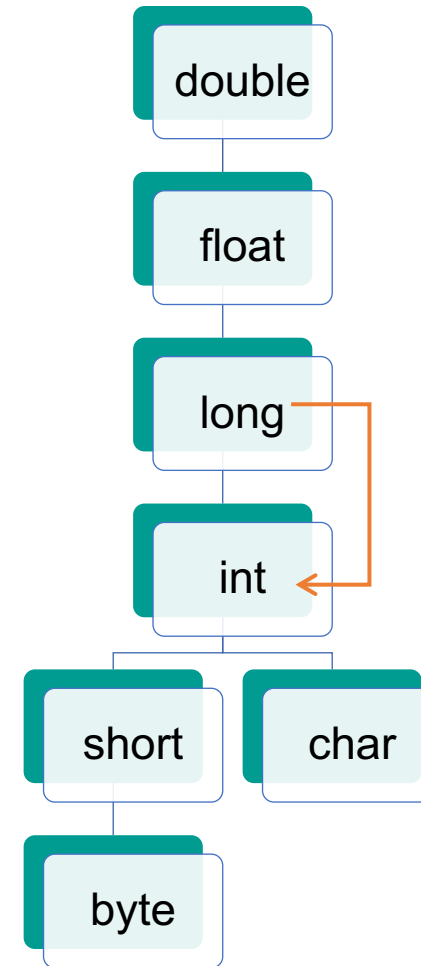
Downcast (Explizit)

- Abwärtskonvertierung muss erzwungen werden
- Nur sinnvoll, wenn **Wert** in niedrigeren/engeren Typ **passt** (dies kann der Compiler nicht prüfen, da erst zur Laufzeit bekannt)

– Java:

```
int intNumber;  
long longNumber = 42L;  
  
intNumber = (int) longNumber;
```

↑
(<Typ>) erzwingt Umwandlung



Explizite Typumwandlung

- Beispiel Abwärtskonvertierung

```
int intNumber;  
long longNumber = 42L;  
  
intNumber = (int) longNumber;
```

- Beispiel Abwärtskonvertierung

```
int intNumber;  
long longNumber = 2_500_000_000L;  
  
intNumber = (int) longNumber;
```

- Was passiert?

jshell>



code03.
Explicit
Downcast

Fehler long → int verhindern

- `int i = Math.toIntExact(long l);`
- Wirft einen Laufzeit-Fehler (mehr zum Thema Exceptions später)
- "Lauter" Fehler: Programm "scheitert", rechnet nicht mit "falschem" Wert weiter
- Math ist Teil der Java-Standard-Bibliothek
- Im Package `java.lang` wie auch `String` (alle Klassen dort sind ohne weiteres Zutun verfügbar)

jshe11>



code03.
MathToInt
Exact

int → char

- `char c = (char) 65; // c = 'A'`
- Erzeugt das Zeichen mit Unicode-Nummer* 65.

* Die ersten 128 Zeichen im Unicode sind mit dem ASCII-Code identisch.

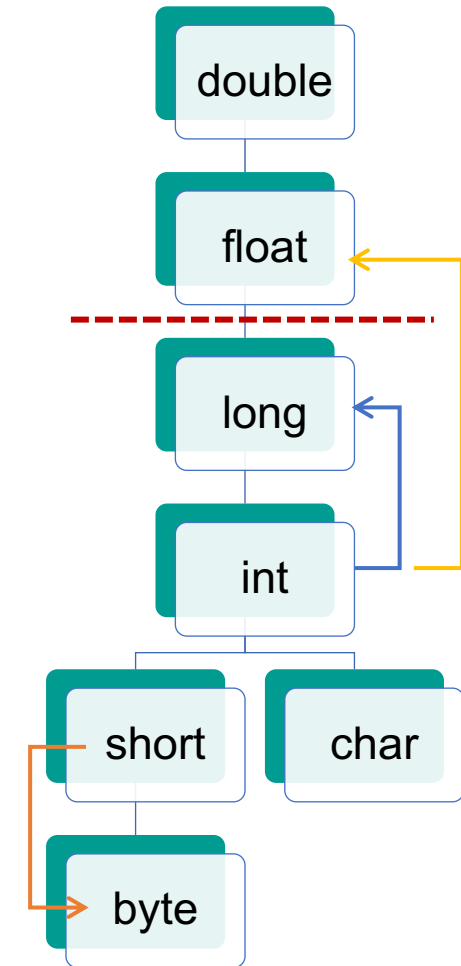
jshe11>



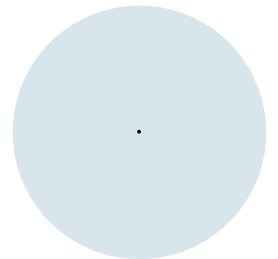
code03.
CharFrom
Int

Zusammenfassung

- **Upcasts** sind implizit durch einfache Zuweisung möglich, z.B. `int` zu `long`
- Vorsicht bei Upcasts, die die rote Linie überschreiten, z.B. `int` zu `float`: Hier kann es zu Präzisionsverlusten kommen
- Vorsicht bei **Downcasts**, denn der Wert kann zu groß für den engeren Typ sein. Downcasts müssen daher mit dem Cast-Operator (`<Typ>`) erzwungen werden, z.B. `short` zu `byte`
- Für die Umwandlung `long` zu `int` am besten `Math.toIntExact(long l)` verwenden



Kommentare



Wozu kommentieren?

- Nützlich zur Erklärung von Programmen
- Helfen anderen und Ihnen in der Zukunft

Hinweis: Kommentare werden vom Compiler ignoriert
→ kein Einfluss auf Geschwindigkeit oder Speicherplatz

Wie kommentieren?

- Zeilenkommentar //
- Alles hinter // wird ignoriert

```
// dies wird vom Compiler ignoriert  
int i = 42; // dies auch
```

- Block-Kommentar /* */
- Alles zwischen /* und */ wird ignoriert, auch über mehrere Zeilen

```
/* Das hier  
   wird alles  
   ignoriert */  
int j = /* und das hier auch */ 42;
```



Was und wie kommentieren?

- **Was?** Das kommentieren, was Ihnen in 3 Monaten nicht mehr klar ist.
 - Achtung: Das können Sie aktuell schlecht einschätzen (das menschliche Gehirn hat ein relativ großes Kurzzeitgedächtnis)
- **Wie?** Nicht beschreiben, was sowieso schon dasteht, sondern Zusatzinformationen liefern

```
So // Deklaration von Summe mit 0 als initialem Wert  
nicht: int sum = 0;  
  
Besser // In dieser Variable soll am Ende die Summe  
so: // aller Zahlen stehen.  
int sum = 0;
```